

DEVOPS MASTER COURSE

Volume 4

Docker and Containerization

From First Container to Secure Production Delivery

Course dimension	Definition
Audience	Complete beginners progressing to professional DevOps and platform engineering practice
Learning method	Understand -> build -> inspect -> break -> recover -> secure -> document
Coverage	Docker Engine, images, containers, networking, storage, Compose, registries, BuildKit, security, CI/CD and operations
Assessment	48 units, practical labs, two capstones, interview preparation and final examination

Run all destructive labs only in a disposable environment. Never experiment on production hosts without authorization, backup and rollback.

How to Use This Volume

Read each unit, reproduce every command, inspect the resulting state, deliberately create one controlled failure, recover it, and record the evidence. A command succeeding is not the same as understanding the system.

Stage	Action	Evidence
Learn	Read the concepts and define unfamiliar terms	Personal notes
Build	Run the laboratory in a disposable environment	Terminal history or screenshots
Inspect	Use docker inspect, logs, stats and network/storage commands	Recorded output
Break	Introduce one controlled error	Failure description
Recover	Restore service safely	Recovery steps
Complete	Answer review questions and mark the tracker	80% or better

Recommended Lab Setup

- Windows 11 with WSL2, Linux, or macOS with a supported Docker installation.
- At least 8 GB RAM and 20 GB free disk space for comfortable labs.
- A Git repository for Dockerfiles, Compose files, scripts and notes.
- A private registry account or local registry for image-distribution exercises.
- Never store real credentials in lab files or image layers.

Course Map

Part	Focus	Units
I	Container fundamentals and Docker Engine	1-8
II	Images and Dockerfile engineering	9-18
III	Runtime, networking and storage	19-28
IV	Compose, registries, security and supply chain	29-39
V	CI/CD, production operations and troubleshooting	40-44
VI	Capstones, interviews and final examination	45-48

Unit 1: Why Containers Exist

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain problems solved by containers.
- Compare physical servers, virtual machines and containers.
- Place containers in the DevOps lifecycle.

Why this matters

Containers improve consistency, packaging and deployment speed by combining application code with its declared runtime dependencies.

Before containers

Applications often behaved differently across developer laptops, test systems and production because operating systems, libraries, configuration and manual setup drifted.

Containers versus virtual machines

- Virtual machines emulate or virtualize hardware and usually run full guest operating systems.
- Containers share the host kernel while isolating processes, filesystems, networks and resource views.
- Containers are lighter, but they are not automatically more secure than virtual machines.

DevOps flow

Source code -> image build -> automated tests -> registry -> deployment platform -> observability -> rollback or promotion.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

1. Write five examples of environment drift.
2. Draw VM and container architecture.
3. List container benefits and risks.

Knowledge check

4. Do containers contain a full kernel?
5. Why are images useful for reproducibility?
6. What does a registry do?

Completion evidence: _____

Unit 2: Linux Foundations Behind Containers

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Understand namespaces and cgroups.
- Explain copy-on-write filesystems.
- Recognize that a container is a constrained process.

Why this matters

Docker is easier to operate when containers are understood as Linux processes with isolation and resource controls.

Namespaces

Namespaces isolate views of process IDs, networks, mounts, hostnames, users and inter-process communication.

Control groups

cgroups account for and limit CPU, memory, processes and other resources.

Layered filesystems

Image layers are read-only. A running container receives a writable layer. Copy-on-write avoids duplicating unchanged data.

Security boundary

The host kernel is shared. Kernel vulnerabilities, excessive capabilities or privileged containers can weaken isolation.

Command laboratory

Inspect namespaces

```
lsns
cat /proc/self/cgroup
```

These host commands expose namespace and cgroup information.

Inspect a process

```
ps -ef
cat /proc/1/status | head
```

PID 1 has special behavior in both hosts and containers.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

7. Inspect host namespaces.
8. Run a container later and compare process views.
9. Explain why shared kernel matters.

Knowledge check

10. What does a PID namespace isolate?
11. What is copy-on-write?
12. Why is privileged mode risky?

Completion evidence: _____

Unit 3: Installing Docker and Validating the Engine

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Install a supported Docker environment.
- Identify client and server components.
- Validate installation safely.

Why this matters

A correct installation prevents later confusion about daemon access, virtualization and compatibility.

Architecture

The docker CLI talks to a Docker daemon over a local socket or remote API. The daemon manages images, containers, networks and volumes.

Desktop and Engine

Docker Desktop packages a managed Linux environment for Windows and macOS. Linux can run Docker Engine directly.

Permissions

Membership in the docker group effectively grants root-equivalent control on many Linux hosts. Treat it as privileged access.

Command laboratory

Validate

```
docker version
docker info
```

version shows client and server. info displays daemon configuration.

Smoke test

```
docker run --rm hello-world
```

The command pulls an image if absent and runs a short container.

Context

```
docker context ls
docker context show
```

Contexts select which daemon endpoint the CLI controls.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

13. Install Docker using the official supported method for your platform.
14. Run the smoke test.
15. Record client version, server version, storage driver and cgroup version.

Knowledge check

- 16. Why can docker group membership be privileged?
- 17. What is the daemon?
- 18. What does a context select?

Completion evidence: _____

Unit 4: Docker CLI Object Model

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Use object-oriented Docker commands.
- List images, containers, networks and volumes.
- Read command help effectively.

Why this matters

Professional operation depends on knowing which Docker object is being manipulated.

Main objects

- Images are immutable templates.
- Containers are runtime instances of images.
- Networks connect containers.
- Volumes store managed persistent data.
- Plugins extend selected subsystems.

Command pattern

`docker <object> <action>`, for example `docker container ls` or `docker image inspect`.

Compatibility aliases

Short forms such as `docker ps` remain common, but explicit object commands improve discoverability.

Command laboratory

List objects

```
docker container ls -a
docker image ls
docker network ls
docker volume ls
```

Build a baseline inventory.

Discover help

```
docker container --help
docker run --help
```

Use built-in help before guessing flags.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

19. Create an inventory script.
20. Compare `docker ps` with `docker container ls`.
21. Use help to find the restart-policy flag.

Knowledge check

- 22. What is the difference between image and container?
- 23. Which object stores persistent data?
- 24. Why use explicit object commands?

Completion evidence: _____

Unit 5: Running and Inspecting Containers

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Create foreground and detached containers.
- Inspect metadata.
- Understand container lifecycle states.

Why this matters

Running containers safely requires explicit control over names, commands, cleanup and observation.

Create versus start

docker create prepares a stopped container. docker start runs an existing container. docker run combines image selection, creation and start.

Foreground and detached

Foreground attaches output to the terminal. Detached mode returns control while the container runs in the background.

Metadata

docker inspect returns authoritative low-level configuration and runtime state.

Command laboratory

Run interactively

```
docker run --rm -it alpine:3.20 sh
```

--rm deletes the stopped container.

Run detached

```
docker run -d --name web nginx:1.27-alpine
```

Use explicit tags rather than latest for controlled work.

Inspect

```
docker container inspect web
```

```
docker container ls
```

Inspect image, command, state and network settings.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

25. Run foreground and detached containers.
26. Inspect the web container.
27. Stop and remove it cleanly.

Knowledge check

28. What does --rm do?

29. When should detached mode be used?

30. What does inspect show?

Completion evidence: _____

Unit 6: Container Lifecycle and Signals

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Stop and kill containers correctly.
- Understand PID 1 and signal handling.
- Configure restart policies.

Why this matters

Graceful shutdown protects requests, files and data integrity.

Lifecycle states

Created, running, paused, restarting, exited and dead are common states.

Stop versus kill

stop sends a termination signal and waits before forcing termination. kill sends a selected signal immediately.

PID 1 behavior

The main process must receive and handle signals and reap child processes. Shell wrappers can accidentally intercept signals.

Restart policy

Policies include no, on-failure, always and unless-stopped. Orchestrators may provide a different restart model.

Command laboratory

Lifecycle

```
docker stop web
docker start web
docker restart web
```

Observe state transitions.

Signal

```
docker kill --signal=TERM web
```

Use a controlled lab.

Restart policy

```
docker update --restart unless-stopped web
docker inspect -f '{{.HostConfig.RestartPolicy.Name}}' web
```

Update and verify policy.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

31. Run a long-lived test process.
32. Compare stop and kill behavior.

33. Test an on-failure restart policy.

Knowledge check

34. Why is graceful shutdown important?

35. What is special about PID 1?

36. When is always inappropriate?

Completion evidence: _____

Unit 7: Executing Commands, Logs and Copying Files

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Use exec appropriately.
- Read logs and timestamps.
- Copy files for diagnosis without treating containers as pets.

Why this matters

Containers should be reproducible, but operators still need controlled runtime diagnosis.

Exec

docker exec starts another process in a running container. It does not modify the image.

Logs

docker logs reads the configured logging driver output, usually stdout and stderr from the main process.

Copy

docker cp copies between host and container, useful for diagnostics but not as a deployment method.

Command laboratory

Exec

```
docker exec -it web sh
```

Minimal images may not include bash.

Logs

```
docker logs --tail 50 --timestamps web
```

```
docker logs -f web
```

Follow only as long as needed.

Copy

```
docker cp web:/etc/nginx/nginx.conf ./nginx.conf
```

Inspect configuration locally.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

37. Generate HTTP requests and observe logs.
38. Use exec to inspect processes.
39. Copy a configuration file and compare it with the image source.

Knowledge check

40. Does exec change the image?
41. Why should apps log to stdout/stderr?
42. Why is docker cp not configuration management?

Completion evidence: _____

Unit 8: Resource Limits and Runtime Controls

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Set CPU and memory limits.
- Understand OOM behavior.
- Use process and file descriptor limits.

Why this matters

One noisy container can degrade every workload on the host unless resources are governed.

Memory

Memory limits constrain usage. Exceeding the limit can trigger out-of-memory termination.

CPU

CPU shares influence relative weight; quotas constrain usable CPU time.

PIDs and ulimits

PID limits reduce fork-bomb risk. ulimits constrain resources such as open files.

Reservations

Some orchestrators distinguish requests/reservations from hard limits; Docker run flags mainly configure runtime constraints.

Command laboratory

Run limited container

```
docker run --rm --memory 128m --cpus 0.5 --pids-limit 100 alpine:3.20 sh -c 'ulimit -a; sleep 5'
```

Inspect limits in a disposable container.

Inspect limits

```
docker inspect -f '{{json .HostConfig.Memory}} {{json .HostConfig.NanoCpus}}' <container>
```

Replace the placeholder.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

43. Run a CPU-limited workload.
44. Run a controlled memory stress test below host risk thresholds.
45. Record exit code and OOM state.

Knowledge check

46. What happens when memory is exhausted?
47. What is a PID limit?
48. Why are limits essential on shared hosts?

Completion evidence: _____

Unit 9: Images, Layers and Digests

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain image layers.
- Differentiate tags and digests.
- Inspect image history.

Why this matters

Images are the deployable supply-chain unit. Tag discipline and digest pinning improve traceability.

Layers

Each build instruction can create a filesystem layer or metadata change. Layers are shared and cached.

Tags

A tag is a mutable human-readable reference such as 1.2.3.

Digests

A digest identifies content. Deploying by digest provides stronger immutability than a tag.

History

Image history helps identify instructions, size contributors and accidental secret exposure.

Command laboratory

Inspect image

```
docker image inspect nginx:1.27-alpine
docker image history nginx:1.27-alpine
```

Review layers and configuration.

Digest

```
docker pull nginx:1.27-alpine
docker image inspect --format '{{index .RepoDigests 0}}' nginx:1.27-alpine
```

The registry digest appears after pull.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

49. Compare two tags of the same image.
50. Record size, layers and digest.
51. Explain why latest is unsafe.

Knowledge check

52. Can a tag move?
53. What does a digest identify?
54. Why do layers improve distribution?

Completion evidence: _____

Unit 10: Your First Dockerfile

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Write a basic Dockerfile.
- Build and run an image.
- Understand build context.

Why this matters

A Dockerfile is executable documentation for a repeatable image build.

Build context

The context is the directory sent or exposed to the builder. Instructions such as COPY can access only permitted context files.

Core instructions

FROM selects a base, WORKDIR sets a directory, COPY adds files, RUN executes build commands, CMD defines default runtime arguments.

Reproducibility

Pin versions, keep the context small and verify the built image.

Command laboratory

Dockerfile

```
cat > Dockerfile <<'EOF'
FROM alpine:3.20
WORKDIR /app
COPY message.txt .
CMD ["cat", "/app/message.txt"]
EOF
echo 'Hello from Volume 4' > message.txt
```

Create a minimal image.

Build and run

```
docker build -t volume4/hello:1.0 .
docker run --rm volume4/hello:1.0
```

The final dot is the build context.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

55. Build the image.
56. Change the message and rebuild.
57. Inspect image history.

Knowledge check

58. What is build context?

59. What does FROM do?

60. Why use an explicit tag?

Completion evidence: _____

Unit 11: Dockerfile Instructions in Depth

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Use COPY, ADD, RUN, ENV, ARG and WORKDIR correctly.
- Understand build-time versus runtime values.
- Avoid shell ambiguity.

Why this matters

Instruction choices affect caching, security, portability and runtime behavior.

COPY versus ADD

Prefer COPY for local files. ADD has extra behaviors such as archive extraction and remote URL handling that can surprise readers.

ARG and ENV

ARG exists during build. ENV becomes image configuration and is available at runtime unless overridden.

WORKDIR

WORKDIR changes the directory for later instructions and runtime defaults.

Labels

LABEL stores metadata such as source repository and version.

Command laboratory

Example

```
cat > Dockerfile <<'EOF'
FROM alpine:3.20
ARG APP_VERSION=dev
ENV APP_HOME=/opt/app
LABEL org.opencontainers.image.version=$APP_VERSION
WORKDIR $APP_HOME
COPY . .
RUN printf '%s\n' "$APP_VERSION" > VERSION
CMD ["cat", "VERSION"]
EOF
```

Do not pass secrets through ARG or ENV.

Build argument

```
docker build --build-arg APP_VERSION=1.2.0 -t volume4/meta:1.2.0 .
```

Inspect labels and environment afterward.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

- 61. Build with two versions.
- 62. Inspect image config.
- 63. Prove ARG is not a runtime variable unless copied into ENV or files.

Knowledge check

- 64. Why prefer COPY?
- 65. Can ARG safely hold secrets?
- 66. What does WORKDIR affect?

Completion evidence: _____

Unit 12: CMD, ENTRYPOINT and Signal-Safe Execution

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Differentiate CMD and ENTRYPOINT.
- Use exec form.
- Design override behavior.

Why this matters

Incorrect process invocation causes shutdown, quoting and argument-handling failures.

CMD

CMD defines default command or default arguments. It is easily replaced at docker run time.

ENTRYPOINT

ENTRYPOINT defines the executable. CMD can supply default arguments to it.

Exec versus shell form

Exec form runs the program directly. Shell form runs through `/bin/sh -c` and may interfere with signals and expansion.

Command laboratory

Exec form

```
ENTRYPOINT ["/usr/local/bin/app"]
CMD ["--config", "/etc/app/config.yaml"]
```

This gives a stable executable with overridable arguments.

Override

```
docker run --rm image:tag --help
docker run --rm --entrypoint sh image:tag
```

Use entrypoint override for diagnosis, not normal operation.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

67. Build a script-based image with exec-form ENTRYPOINT.
68. Override CMD arguments.
69. Observe PID 1 using `ps`.

Knowledge check

70. Which is replaced by trailing docker run arguments?
71. Why prefer exec form?
72. When might `--entrypoint` be useful?

Completion evidence: _____

Unit 13: Layer Caching and Build Performance

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Predict cache reuse.
- Order instructions for efficient builds.
- Use cache mounts conceptually.

Why this matters

Fast, deterministic builds increase developer productivity and reduce CI cost.

Cache key

The builder reuses results when instruction inputs and relevant metadata match.

Ordering

Copy stable dependency manifests and install dependencies before copying frequently changing source.

Invalidation

A changed earlier layer invalidates later dependent layers.

BuildKit

BuildKit supports parallelism, improved caching and advanced mounts.

Command laboratory

Observe cache

```
docker build --progress=plain -t volume4/cache:1 .
docker build --progress=plain -t volume4/cache:2 .
```

The second build should reuse eligible steps.

No cache

```
docker build --no-cache -t volume4/cache:fresh .
```

Use deliberately; it is not a universal fix.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

73. Measure two consecutive builds.
74. Change only source and identify rebuilt layers.
75. Reorder a poor Dockerfile.

Knowledge check

76. Why copy dependency manifests first?
77. What invalidates later layers?
78. When is --no-cache justified?

Completion evidence: _____

Unit 14: Multi-Stage Builds

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Separate build and runtime stages.
- Reduce image size and attack surface.
- Copy only required artifacts.

Why this matters

Production images should not contain compilers, package managers and test dependencies unless required at runtime.

Stages

Each FROM starts a stage. Artifacts can be copied from earlier stages.

Benefits

Smaller images, fewer vulnerabilities, less intellectual-property leakage and cleaner runtime.

Named stages

Naming stages makes COPY --from readable and resilient to reordering.

Command laboratory

Example

```
cat > Dockerfile <<'EOF'
FROM golang:1.23-alpine AS build
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -o /out/app ./cmd/app
FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/app /app
USER nonroot:nonroot
ENTRYPOINT ["/app"]
EOF
```

Adapt the example to an actual Go project.

Target stage

```
docker build --target build -t volume4/app-build .
```

Useful for debugging a build stage.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

79. Create a two-stage build for any compiled or bundled sample.
80. Compare size with a single-stage image.
81. Inspect runtime contents.

Knowledge check

82. Why are multi-stage builds safer?

83. What does COPY --from do?

84. Why use a non-root runtime?

Completion evidence: _____

Unit 15: Base Image Selection

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Evaluate base images.
- Understand slim, Alpine and distroless tradeoffs.
- Pin trusted sources.

Why this matters

The base image determines much of the final image's operating system content, compatibility and vulnerability surface.

Selection criteria

Compatibility, support lifecycle, patch cadence, package availability, provenance, size and operational tooling.

Alpine

Small and musl-based. Some applications or native dependencies expect glibc and may need extra validation.

Distroless

Contains runtime essentials without a shell or package manager. It reduces surface but changes debugging methods.

Scratch

Empty base suitable for self-contained static binaries and carefully prepared files.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

85. Build the same simple service on two bases.
86. Compare size and installed utilities.
87. Document compatibility and support tradeoffs.

Knowledge check

88. Is smaller always safer?
89. What compatibility issue can Alpine introduce?
90. How do you debug distroless images?

Completion evidence: _____

Unit 16: Package Installation and Image Hygiene

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Install packages reproducibly.
- Remove transient cache.
- Combine related operations safely.

Why this matters

Poor package practices create stale, oversized and vulnerable images.

Versioning

Use supported repositories and pin where the ecosystem and maintenance policy permit.

Cache cleanup

Package metadata and temporary files should not remain in runtime layers.

Single-layer cleanup

Deleting files in a later layer does not remove them from previous layers.

Upgrades

Blind full upgrades during every build may reduce reproducibility. Rebuild frequently from maintained patched bases.

Command laboratory

Debian pattern

```
RUN apt-get update \  
&& apt-get install -y --no-install-recommends ca-certificates curl \  
&& rm -rf /var/lib/apt/lists/*
```

Keep update and install in one instruction.

Alpine pattern

```
RUN apk add --no-cache ca-certificates curl  
--no-cache avoids persistent package index.
```

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

91. Build examples using Debian and Alpine.
92. Inspect image history and size.
93. Explain rebuild-based patching.

Knowledge check

94. Why combine apt-get update and install?
95. Does deleting later shrink older layers?

96. Why rebuild images regularly?

Completion evidence: _____

Unit 17: .dockerignore and Build Context Security

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Reduce build context.
- Prevent accidental file inclusion.
- Audit image contents.

Why this matters

Sensitive or irrelevant files in build context can leak into images or remote builders and slow builds.

.dockerignore

Rules exclude paths from context, similar in syntax to .gitignore but with Docker-specific behavior.

Sensitive files

Exclude .git, credentials, local environment files, test output and large artifacts unless deliberately needed.

COPY discipline

Use explicit COPY paths rather than copying the entire repository without review.

Command laboratory

Example ignore

```
cat > .dockerignore <<'EOF'
.git
.env
*.pem
*.key
node_modules
coverage
Dockerfile*
README.md
EOF
```

Do not rely on ignore rules as the only secret control.

Inspect context

```
docker build --progress=plain -t volume4/context:1 .
```

Review context-transfer size.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

97. Create a deliberately large irrelevant file.
98. Measure context before and after .dockerignore.
99. Audit the final image filesystem.

Knowledge check

- 100. Can ignored secrets still exist in Git history?
- 101. Why avoid COPY . . ?
- 102. What is sent to a remote builder?

Completion evidence: _____

Unit 18: BuildKit Advanced Features

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Use secrets without persisting them.
- Use SSH and cache mounts.
- Understand build outputs and provenance.

Why this matters

BuildKit enables faster and safer builds than legacy patterns.

Secret mounts

Secrets are exposed temporarily to a RUN instruction and are not intended to become image layers.

SSH mounts

Builds can access private Git dependencies through forwarded SSH agents without copying keys.

Cache mounts

Package-manager caches can persist across builds without entering the final image.

Build metadata

Modern build systems can produce provenance and software-bill-of-material information.

Command laboratory

Enable syntax

```
\# syntax=docker/dockerfile:1
```

Place at the first line of the Dockerfile.

Secret mount

```
RUN --mount=type=secret,id=npnrc,target=/root/.npnrc npm ci
```

Supply with `docker build --secret id=npnrc,src=.npnrc`.

Cache mount

```
RUN --mount=type=cache,target=/root/.cache/pip pip install -r requirements.txt
```

The cache accelerates builds without being copied into the image.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

103. Use a harmless secret file and prove it is absent afterward.
104. Use a cache mount.
105. Inspect build metadata where supported.

Knowledge check

106. Why is ARG unsafe for secrets?

107. Does a secret mount persist?

108. What is a cache mount for?

Completion evidence: _____

Unit 19: Container Filesystems and Writable Layers

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Explain ephemeral container writes.
- Inspect filesystem changes.
- Design immutable runtime behavior.

Why this matters

Data written only to the container layer disappears when the container is replaced.

Writable layer

Runtime changes are stored in a thin writable layer above image layers.

Ephemeral design

Containers should be replaceable. Persistent state belongs in volumes or external services.

Inspection

`docker diff` lists added, changed and deleted paths relative to the image.

Command laboratory

Create change

```
docker run -d --name fs-lab alpine:3.20 sh -c 'echo runtime > /tmp/data; sleep 300'
docker diff fs-lab
```

Observe filesystem changes.

Replace

```
docker rm -f fs-lab
docker run --rm alpine:3.20 cat /tmp/data
```

The second command fails because the runtime file was not in the image.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

109. Write data in a container layer.
110. Commit the container only as a demonstration, not a workflow.
111. Replace the container and explain the result.

Knowledge check

112. Why is container-local data ephemeral?
113. What does `docker diff` show?
114. Why is `docker commit` discouraged?

Completion evidence: _____

Unit 20: Volumes and Persistent Data

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Create and mount named volumes.
- Inspect volume ownership and lifecycle.
- Back up and restore data.

Why this matters

Stateful containers require explicit persistence and recovery planning.

Named volumes

Docker manages their location and lifecycle separately from containers.

Mount points

Mounting over a non-empty image directory hides the original path contents while mounted.

Lifecycle

Removing a container does not automatically remove named volumes unless explicitly requested.

Backup

A backup must include consistency controls, verification and restore testing.

Command laboratory

Create volume

```
docker volume create appdata
docker run --rm -v appdata:/data alpine:3.20 sh -c 'echo persistent > /data/value'
docker run --rm -v appdata:/data alpine:3.20 cat /data/value
```

The second container sees persisted data.

Inspect

```
docker volume inspect appdata
```

Location details are implementation-specific.

Backup

```
docker run --rm -v appdata:/data:ro -v "$PWD":/backup alpine:3.20 tar czf /backup/appdata.tgz -C /data .
```

Coordinate application consistency first.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

115. Create a volume and persist data.
116. Back it up and restore to a new volume.
117. Verify checksums or contents.

Knowledge check

- 118. Who manages a named volume?
- 119. What happens when a mount hides image files?
- 120. Why test restores?

Completion evidence: _____

Unit 21: Bind Mounts and Host Integration

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Use bind mounts safely.
- Understand host path coupling.
- Control read-only access.

Why this matters

Bind mounts are powerful for development and configuration, but tightly couple containers to host paths and permissions.

Bind semantics

A host file or directory is mounted directly into the container.

Use cases

Source-code development, controlled configuration files, local certificates and host integration.

Risks

Containers may modify host files, inherit host-specific paths and expose sensitive content.

Read-only

Use read-only mounts whenever the container does not need to write.

Command laboratory

Read-only mount

```
docker run --rm --mount type=bind,src="$PWD",dst=/work,readonly alpine:3.20 ls -la /work
```

--mount is more explicit than -v.

Config file

```
docker run --rm --mount type=bind,src="$PWD/nginx.conf",dst=/etc/nginx/nginx.conf,readonly nginx:1.27-alpine
nginx -t
```

Create a valid configuration first.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

121. Mount a source directory read-only.
122. Attempt a write and observe failure.
123. Compare bind mount and named volume use cases.

Knowledge check

124. Why are bind mounts less portable?
125. What can happen with writable host mounts?
126. When should readonly be used?

Completion evidence: _____

Unit 22: tmpfs and In-Memory Data

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Use tmpfs mounts.
- Recognize suitable temporary workloads.
- Understand memory and persistence implications.

Why this matters

Some sensitive or temporary data should never be written to persistent disk layers.

tmpfs

A tmpfs mount stores data in memory and is removed when the container stops.

Use cases

Temporary caches, runtime scratch space and selected secret-like transient files.

Limits

tmpfs consumes memory and can contribute to memory pressure. It is not a durable store.

Command laboratory

Run tmpfs

```
docker run --rm --tmpfs /run:size=64m alpine:3.20 sh -c 'df -h /run; echo temp > /run/value; cat /run/value'
```

The data disappears with the container.

Explicit mount

```
docker run --rm --mount type=tmpfs,dst=/tmp,tmpfs-size=33554432 alpine:3.20 df -h /tmp
```

Set an appropriate size.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

127. Create tmpfs data.
128. Stop the container and prove it is gone.
129. Document a valid and invalid tmpfs use case.

Knowledge check

130. Is tmpfs persistent?
131. Where does its capacity come from?
132. Can tmpfs replace a database volume?

Completion evidence: _____

Unit 23: Docker Networking Fundamentals

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Understand network namespaces and virtual Ethernet.
- Use the default bridge.
- Inspect container addresses and routes.

Why this matters

Container connectivity is implemented through Linux networking primitives and Docker-managed rules.

Default bridge

Containers attached to the default bridge receive isolated addresses and can reach external networks through host forwarding and translation.

User-defined networks

User-defined bridges provide better DNS-based name resolution and isolation.

Published ports

Publishing maps host addresses and ports to container ports. EXPOSE alone does not publish anything.

Command laboratory

Inspect

```
docker network inspect bridge
```

Review subnet, gateway and attached containers.

Run and inspect

```
docker run -d --name netlab alpine:3.20 sleep 300
docker exec netlab ip addr
docker exec netlab ip route
```

The image may need networking tools depending on base.

Publish

```
docker run -d --name web -p 127.0.0.1:8080:80 nginx:1.27-alpine
```

Binding to loopback limits host exposure.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

133. Inspect default bridge.
134. Publish to loopback and test curl.
135. Compare host-wide 0.0.0.0 binding.

Knowledge check

136. Does EXPOSE publish a port?

137. What does -p do?

138. Why bind to 127.0.0.1 in some cases?

Completion evidence: _____

Unit 24: User-Defined Networks and DNS

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Create isolated networks.
- Use container-name DNS.
- Connect and disconnect containers.

Why this matters

Application tiers should communicate through controlled networks rather than broad default connectivity.

Embedded DNS

On user-defined networks, Docker provides name resolution for attached containers.

Multiple networks

A container can join multiple networks, acting as a boundary only if the application and host controls are designed carefully.

Isolation

Networks reduce accidental reachability but are not a complete security policy.

Command laboratory

Create network

```
docker network create appnet
```

Docker selects a subnet unless specified.

Name resolution

```
docker run -d --name api --network appnet nginx:1.27-alpine
docker run --rm --network appnet alpine:3.20 wget -q0- http://api
```

The client resolves api through Docker DNS.

Connect

```
docker network connect appnet web
docker network disconnect appnet web
```

Inspect before and after.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

139. Create frontend and backend networks.
140. Attach a proxy to both and database only to backend.
141. Validate allowed and blocked paths.

Knowledge check

142. What provides DNS on user-defined networks?

143. Can one container join multiple networks?

144. Is network separation alone sufficient security?

Completion evidence: _____

Unit 25: Port Publishing and Exposure

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Publish TCP and UDP ports deliberately.
- Avoid accidental public exposure.
- Diagnose host-to-container forwarding.

Why this matters

Incorrect port publishing can expose internal administration interfaces to every network.

Syntax

Host IP:host port:container port. Omitting host IP often binds all host interfaces.

Random ports

-P publishes declared EXPOSE ports to random host ports.

IPv6

Binding behavior depends on daemon and host configuration. Verify actual listeners.

Firewall interaction

Docker may manage host firewall/NAT rules. Host policy must account for Docker behavior.

Command laboratory

Loopback publish

```
docker run -d --name localweb -p 127.0.0.1:8081:80 nginx:1.27-alpine
```

Only local clients should reach it.

Inspect mapping

```
docker port localweb
ss -lntp | grep 8081
```

Validate on the host.

UDP example

```
docker run --rm -p 127.0.0.1:9999:9999/udp alpine:3.20 sh -c 'sleep 30'
```

A listener is still required inside for real service.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

145. Compare loopback and all-interface bindings.
146. Use ss and docker port.
147. Write an exposure review checklist.

Knowledge check

- 148. What happens when host IP is omitted?
- 149. Does mapping create an application listener?
- 150. Why verify with ss?

Completion evidence: _____

Unit 26: Network Drivers: Bridge, Host, None and Overlay

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Compare built-in drivers.
- Choose the correct isolation model.
- Understand overlay prerequisites.

Why this matters

Different runtime environments require different network semantics.

Bridge

Common single-host container networking.

Host

The container shares the host network namespace, reducing isolation and eliminating normal port mapping.

None

No external interface is configured except loopback.

Overlay

Connects workloads across multiple hosts, typically through an orchestrator such as Swarm.

Command laboratory

Host mode

```
docker run --rm --network host nginx:1.27-alpine
```

Use only in a disposable Linux lab; Docker Desktop behavior differs.

None

```
docker run --rm --network none alpine:3.20 ip addr
```

Only loopback should be present.

Drivers

```
docker network ls
```

Inspect available drivers.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

151. Test bridge and none modes.
152. Document host-mode risks.
153. Explain when overlay is required.

Knowledge check

154. Why does `-p` not work normally with host mode?

155. What connectivity does none provide?

156. Is overlay typically single-host?

Completion evidence: _____

Unit 27: Container Networking Troubleshooting

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Diagnose DNS, route, listener and publish failures.
- Use layered tests.
- Capture evidence.

Why this matters

Container outages often result from confusing application bind addresses, container ports, host mappings and DNS.

Method

Check container state -> logs -> process -> listener -> container-local request -> network attachment -> DNS -> host publish -> firewall -> upstream.

Bind address

An application listening only on 127.0.0.1 inside its container may be unreachable through the container interface.

Tools

docker inspect, docker exec, ss, ip, getent, curl, nc and tcpdump.

Command laboratory

Baseline

```
docker ps
docker logs <container>
docker inspect <container>
docker exec <container> ss -lntp
```

Replace placeholders.

DNS test

```
docker run --rm --network appnet alpine:3.20 getent hosts api
```

Use a network-aware image.

Host test

```
curl -v http://127.0.0.1:8080
ss -lntp | grep 8080
```

Separate container service from host mapping.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

157. Break an application by binding it to the wrong address.
158. Diagnose and fix it.
159. Write a layer-by-layer incident note.

Knowledge check

- 160. Why can localhost inside a container be misleading?
- 161. What should be tested before firewall changes?
- 162. Which command shows network attachment?

Completion evidence: _____

Unit 28: Storage Troubleshooting and Data Recovery

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Diagnose mount errors.
- Handle permissions and ownership.
- Plan safe recovery.

Why this matters

Storage mistakes can destroy state or make healthy applications appear broken.

Common failures

Wrong host path, hidden image content, UID/GID mismatch, read-only filesystem, full disk, inode exhaustion and corrupted application data.

Permissions

The numeric identity inside the container must be able to access mounted data. Names may differ while UIDs match.

Recovery order

Stop writes, preserve evidence, snapshot or back up, repair on a copy where possible, validate restore, then resume.

Command laboratory

Inspect mounts

```
docker inspect -f '{{json .Mounts}}' <container>
```

Review source, destination, type and read/write mode.

Disk

```
df -h
df -i
docker system df
```

Check bytes, inodes and Docker usage.

Ownership

```
docker exec <container> id
ls -ln <host-path>
```

Compare numeric IDs.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

163. Create a permission failure with a bind mount.
164. Diagnose using numeric IDs.
165. Repair without chmod 777.

Knowledge check

- 166. Why is `chmod 777` a poor fix?
- 167. What does `df -i` show?
- 168. Why `stop` writes before recovery?

Completion evidence: _____

Unit 29: Docker Compose Fundamentals

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Define multi-container applications.
- Use services, networks and volumes.
- Operate a project lifecycle.

Why this matters

Compose provides a declarative local and single-host application model useful for development, testing and selected production scenarios.

Compose model

A YAML file defines services, networks, volumes, configs and secrets.

Project

Compose groups resources under a project name.

Declarative intent

The file describes desired configuration; docker compose reconciles by creating or replacing resources as needed.

Command laboratory

Example

```
cat > compose.yaml <<'EOF'
services:
  web:
    image: nginx:1.27-alpine
    ports:
      - "127.0.0.1:8080:80"
EOF
```

Use current Compose specification syntax.

Lifecycle

```
docker compose up -d
docker compose ps
docker compose logs
docker compose down
```

down removes project containers and default network.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

169. Create and start the project.
170. Inspect generated names and network.
171. Modify the port and reconcile.

Knowledge check

- 172. What is a Compose project?
- 173. Does down remove named volumes by default?
- 174. Why bind to loopback?

Completion evidence: _____

Unit 30: Compose Services, Dependencies and Health Checks

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Configure service dependencies.
- Write meaningful health checks.
- Understand readiness limitations.

Why this matters

Startup ordering alone does not guarantee that a dependency is ready to serve.

depends_on

It controls dependency order and can use health conditions in supported Compose behavior, but applications still need retry logic.

Healthcheck

A health check should verify useful service capability with low overhead.

Resilience

Applications should tolerate dependency restarts and temporary unavailability.

Command laboratory

Healthcheck

```
services:
  web:
    image: nginx:1.27-alpine
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://127.0.0.1/"]
      interval: 10s
      timeout: 3s
      retries: 3
```

Validate the tool exists in the image.

Inspect

```
docker compose ps
docker inspect -f '{{json .State.Health}}' <container>
```

Observe transitions.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

175. Add a health check.
176. Create a dependent service with retries.
177. Break the endpoint and observe health.

Knowledge check

- 178. Is started equal to ready?
- 179. Why do applications still need retry logic?
- 180. What makes a good health check?

Completion evidence: _____

Unit 31: Compose Configuration, Environment and Profiles

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Use environment interpolation.
- Separate configuration from images.
- Use profiles for optional services.

Why this matters

Configuration must vary safely by environment without rebuilding application code.

Interpolation

Compose can substitute host environment and .env values into the model.

env_file

Loads variables into a service; it is not a secure secret store.

Profiles

Profiles enable optional services such as debugging or observability components.

Validation

docker compose config renders the effective model and catches many errors.

Command laboratory

Render config

```
docker compose config
```

Review before deployment.

Profile

```
docker compose --profile debug up -d
```

Only services in the selected profile are activated alongside unprofiled services.

Environment

```
APP_PORT=8085 docker compose config
```

Test interpolation.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

181. Create dev and test variable sets.
182. Add a debug profile.
183. Verify no secret appears in committed files.

Knowledge check

184. Is .env a secret manager?

185. What does compose config do?

186. What are profiles for?

Completion evidence: _____

Unit 32: Compose Networking and Storage

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Design service networks.
- Use named volumes.
- Limit unnecessary reachability.

Why this matters

Compose can encode application topology and data ownership.

Default network

Services join a project network and resolve each other by service name.

Custom networks

Separate edge, application and data tiers.

Named volumes

Compose creates and labels managed volumes. External volumes must already exist.

Mount syntax

Long syntax makes type, source, target and read-only intent explicit.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

187. Build a three-service topology.
188. Place database only on backend.
189. Persist database data and test container replacement.

Knowledge check

190. How do services resolve each other?
191. What does external volume mean?
192. Why use separate networks?

Completion evidence: _____

Unit 33: Registries and Image Distribution

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Push and pull images.
- Understand repository naming.
- Operate a local registry.

Why this matters

Registries distribute immutable artifacts between build and runtime environments.

Reference format

registry/namespace/repository:tag or registry/namespace/repository@digest.

Authentication

Use credential helpers, short-lived tokens and least privilege.

Local registry

A local registry is useful for labs and air-gapped patterns, but production requires TLS, authentication, retention and backup.

Garbage collection

Registry storage cleanup must respect manifests and retention policy.

Command laboratory

Local registry

```
docker run -d --restart=always -p 127.0.0.1:5000:5000 --name registry registry:2
```

For local lab use.

Tag and push

```
docker tag volume4/hello:1.0 localhost:5000/volume4/hello:1.0
docker push localhost:5000/volume4/hello:1.0
```

Verify by removing local tag and pulling.

Login

```
docker login <registry>
```

Use secure credential handling.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

193. Run a local registry.
194. Push and pull an image.
195. Record digest before and after.

Knowledge check

- 196. What is an image repository?
- 197. Why use credential helpers?
- 198. What production controls does a registry need?

Completion evidence: _____

Unit 34: Image Tagging and Release Promotion

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Design immutable tags.
- Promote by digest.
- Avoid rebuilding per environment.

Why this matters

A tested artifact should move through environments without changing its contents.

Tag strategy

Use human version tags, commit identifiers and immutable digest records.

Promotion

Promote or retag the same digest after policy checks rather than rebuilding source for staging and production.

latest

latest is only an ordinary mutable tag and provides poor traceability.

Rollback

Rollback selects a previously approved digest.

Command laboratory

Tag

```
docker tag app:build-abc registry.example/app:1.4.0
docker tag app:build-abc registry.example/app:git-abc123
```

Examples only.

Digest deploy

```
docker pull registry.example/app@sha256:<digest>
```

Use the exact approved digest.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

199. Write a tagging standard.
200. Simulate dev -> staging -> production promotion with a local registry.
201. Perform rollback by digest.

Knowledge check

202. Why not rebuild per environment?
203. What is wrong with latest?
204. What is the strongest immutable reference?

Completion evidence: _____

Unit 35: Container Security Foundations

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Apply least privilege.
- Reduce capabilities.
- Use read-only filesystems and secure defaults.

Why this matters

Containers do not remove the need for host, application and supply-chain security.

Non-root

Run as a dedicated numeric user where possible.

Capabilities

Linux capabilities split root powers. Drop all and add only required capabilities.

Filesystem

Read-only root filesystems reduce persistence opportunities; writable paths can be explicit mounts.

Privileges

Avoid privileged, host PID/network, Docker socket mounts and sensitive host paths.

Command laboratory

Hardened run

```
docker run --rm --read-only --tmpfs /tmp --cap-drop ALL --security-opt no-new-privileges:true alpine:3.20 id
```

Some applications need selected writable paths or capabilities.

Inspect user

```
docker image inspect -f '{{.Config.User}}' <image>
```

Empty often means root by default.

Capabilities

```
docker inspect -f '{{json .HostConfig.CapDrop}}' <container>
```

Verify runtime settings.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

205. Run an image as non-root.
206. Make root filesystem read-only.
207. Document required exceptions.

Knowledge check

208. Why is privileged dangerous?

209. What does no-new-privileges do?

210. Why drop capabilities?

Completion evidence: _____

Unit 36: Docker Daemon and Host Security

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Protect the Docker API.
- Understand socket risk.
- Harden the host.

Why this matters

Control of the Docker daemon can often become control of the host.

Socket

Mounting `/var/run/docker.sock` into a container grants powerful daemon access.

Remote API

Never expose an unauthenticated Docker TCP API. Use mutual TLS or a secure remote access mechanism where remote administration is required.

Host hardening

Patch kernel and Docker, restrict administrators, audit daemon configuration, isolate workloads and monitor events.

Rootless mode

Rootless Docker reduces daemon privilege but has feature and performance tradeoffs.

Command laboratory

Inspect socket

```
ls -l /var/run/docker.sock
```

Review ownership and permissions.

Events

```
docker events --since 10m
```

Useful for auditing runtime changes.

Rootless context

```
docker context ls
```

Rootless installations may expose a separate socket/context.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

211. Write a daemon access-control policy.
212. Identify users with Docker access.
213. Create an incident checklist for an exposed API.

Knowledge check

- 214. Why is the Docker socket sensitive?
- 215. Can group access imply root-equivalent control?
- 216. What is rootless mode?

Completion evidence: _____

Unit 37: Image Vulnerability Scanning and SBOMs

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Understand vulnerability scanning.
- Generate or consume SBOMs.
- Prioritize remediation intelligently.

Why this matters

Image scanning identifies known vulnerable components, but results require context and lifecycle management.

Scanner limits

Scanners depend on package inventories and vulnerability databases. They can miss application flaws and may report false positives.

SBOM

A software bill of materials lists components and versions for transparency and incident response.

Prioritization

Consider exploitability, exposure, runtime reachability, severity, available fix and business impact.

Rebuild

Patch by updating dependencies or base images and rebuilding; do not patch running containers manually.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

217. Scan a lab image using an approved scanner available in your environment.
218. Export an SBOM.
219. Choose three findings and write remediation decisions.

Knowledge check

220. Does zero scanner findings mean secure?
221. What does an SBOM contain?
222. Why rebuild instead of patching a live container?

Completion evidence: _____

Unit 38: Secrets Management

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Keep secrets out of images and source.
- Inject secrets at runtime.
- Rotate exposed credentials.

Why this matters

Secrets embedded in image layers or environment dumps can spread to registries, caches and logs.

Unsafe patterns

Dockerfile ENV, build ARG, copied credential files, committed .env and command-line secrets visible in process or history.

Runtime injection

Use orchestrator secret stores, mounted files, short-lived identity tokens or approved external secret managers.

Rotation

If a real secret enters an image or registry, revoke or rotate first, then remove affected artifacts and investigate access.

Logging

Applications must avoid printing secrets.

Command laboratory

Inspect configuration

```
docker inspect <container>
```

Environment variables may be visible to authorized daemon users.

BuildKit secret

```
docker build --secret id=mysecret,src=secret.txt .
```

Use with RUN --mount=type=secret.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

223. Create a dummy secret and inject it as a mounted file.
224. Prove it is absent from image history.
225. Write an exposure-response runbook.

Knowledge check

226. Why is ENV unsafe for secrets?
227. What is the first response to exposure?
228. Can daemon administrators inspect container environment?

Completion evidence: _____

Unit 39: Supply-Chain Security and Image Trust

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Verify image source.
- Pin critical dependencies.
- Understand signing and provenance.

Why this matters

Compromise can occur before runtime through malicious bases, dependencies, build systems or registries.

Trusted sources

Use maintained official or approved images and internal mirrors where required.

Pinning

Pin by digest for critical reproducibility, while maintaining a process to update and rebuild.

Signing

Signatures associate an artifact with a trusted identity and policy. Verification must be enforced at deployment.

Provenance

Build attestations describe where, how and from which source an image was produced.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

- 229. Create an image-source approval checklist.
- 230. Record base image digest.
- 231. Design a policy that rejects unsigned or unapproved images.

Knowledge check

- 232. Does a signature prove the software is bug-free?
- 233. Why enforce verification?
- 234. What is provenance?

Completion evidence: _____

Unit 40: Docker in CI Pipelines

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Build images in CI.
- Use cache safely.
- Publish only validated artifacts.

Why this matters

CI converts source commits into tested, traceable images.

Pipeline stages

Checkout -> dependency and unit tests -> image build -> image tests -> scan -> sign/attest -> push -> deploy or promote.

Credentials

Use short-lived registry credentials and protected environments.

Cache

Cache can accelerate builds but must not leak secrets or incorrectly reuse untrusted content.

Traceability

Label images with source revision and record digest as pipeline output.

Command laboratory

Local CI-like build

```
docker build --pull --label org.opencontainers.image.revision=$(git rev-parse HEAD) -t app:ci .
```

Use in a Git repository.

Test image

```
docker run --rm app:ci <test-command>
```

Replace the command.

Digest record

```
docker image inspect app:ci --format '{{.Id}}'
```

Registry digest is recorded after push.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

- 235. Write a CI workflow outline.
- 236. Build and test an image.
- 237. Fail the pipeline on a deliberate test error.

Knowledge check

- 238. Why use --pull selectively?
- 239. When should an image be pushed?
- 240. What traceability metadata is useful?

Completion evidence: _____

Unit 41: Testing Container Images

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Test image structure and behavior.
- Validate health and shutdown.
- Use ephemeral integration environments.

Why this matters

An image that builds successfully can still fail at runtime.

Test levels

Lint Dockerfile, test filesystem and metadata, run unit tests, start container, test health endpoint, test signal handling, scan and run integration tests.

Runtime contract

Validate user, ports, read-only behavior, required files, configuration and exit codes.

Ephemeral environments

Compose can create disposable integration stacks for each CI run.

Command laboratory

Metadata assertions

```
test "$(docker image inspect -f '{{.Config.User}}' app:test)" != ""
```

Adapt to expected non-root user.

Startup test

```
cid=$(docker run -d app:test); sleep 2; docker logs "$cid"; docker stop -t 10 "$cid"; docker rm "$cid"
```

Add actual health checks.

Compose test

```
docker compose up -d --wait
```

```
docker compose down -v
```

--wait support depends on Compose version.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

241. Write five image assertions.
242. Test graceful shutdown.
243. Run an ephemeral integration stack.

Knowledge check

244. Why test the built image rather than source only?
245. What should shutdown testing verify?

246. Why destroy integration resources?

Completion evidence: _____

Unit 42: Production Deployment Patterns

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Understand immutable deployment.
- Compare rolling, blue-green and canary strategies.
- Plan rollback.

Why this matters

Production delivery must minimize risk and provide fast recovery.

Immutable deployment

Deploy a new image digest by replacing containers rather than editing them.

Rolling

Replace instances gradually while maintaining capacity.

Blue-green

Maintain old and new environments and switch traffic.

Canary

Send limited traffic to the new version and evaluate signals.

Rollback

Rollback must include application, configuration and data compatibility.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

247. Design all three strategies for a web service.
248. Define success and abort metrics.
249. Write a rollback checklist including database compatibility.

Knowledge check

250. Why avoid editing live containers?
251. What is a canary?
252. Why can database changes block rollback?

Completion evidence: _____

Unit 43: Observability for Containers

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Collect logs, metrics and traces.
- Use container metadata.
- Avoid local-only observability.

Why this matters

Containers are ephemeral, so operational data must leave the container before it disappears.

Logs

Applications should emit structured logs to stdout/stderr. The platform forwards them to durable storage.

Metrics

Track host, daemon, container and application metrics such as CPU, memory, restarts, latency, errors and saturation.

Tracing

Distributed tracing connects requests across services.

Metadata

Include service, version, environment, container or task identity and correlation IDs.

Command laboratory

Stats

```
docker stats --no-stream
```

Observe current resource use.

Events

```
docker events --since 5m
```

Correlate lifecycle changes.

Disk use

```
docker system df -v
```

Review image, container, volume and cache consumption.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

253. Generate load and observe stats.
254. Create structured application logs.
255. Define a dashboard and alert set.

Knowledge check

- 256. Why should logs leave the container?
- 257. What does restart count indicate?
- 258. What metadata helps correlate incidents?

Completion evidence: _____

Unit 44: Production Troubleshooting and Maintenance

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Use a disciplined incident workflow.
- Clean unused resources safely.
- Diagnose build and runtime failures.

Why this matters

Random cleanup and restarts can destroy evidence and worsen outages.

Incident method

Confirm impact and scope, review recent changes, collect state and logs, test layers, mitigate reversibly, validate, monitor and document.

Disk pressure

Unused images, stopped containers, build cache and volumes can consume storage.

Prune safety

Prune commands delete unused resources and can remove recovery material or valuable cache. Inspect first.

Common failures

Crash loops, OOM kills, unhealthy status, permission errors, DNS failures, port conflicts and full disks.

Command laboratory

Triage

```
date -Is
docker ps -a
docker events --since 30m
docker system df
docker info
```

Capture before changing state.

Inspect failure

```
docker inspect <container>
docker logs --tail 200 --timestamps <container>
docker stats --no-stream <container>
```

Replace placeholder.

Preview cleanup

```
docker system df -v
docker image prune --filter until=168h
```

Review impact; avoid broad -a or volume pruning without ownership checks.

How companies use this

Professional teams encode this practice in source-controlled standards, CI policies, image registries, automated checks, least-privilege access and documented rollback.

Common mistakes and prevention

- Do not use production hosts for experiments.
- Do not rely on mutable tags or undocumented manual changes.
- Inspect state and preserve evidence before destructive cleanup or restart.

Hands-on lab

- 259. Simulate a crash loop.
- 260. Diagnose an OOM or port conflict.
- 261. Write a maintenance and retention policy.

Knowledge check

- 262. Why capture evidence before restart?
- 263. What can prune delete?
- 264. What is the difference between mitigation and root cause?

Completion evidence: _____

Unit 45: Capstone 1: Production-Grade Container Image

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Build a secure optimized image.
- Test and document its contract.
- Create release evidence.

Why this matters

This capstone combines Dockerfile engineering, security, testing and traceability.

Scenario

Containerize a small API or web application for production.

Mandatory deliverables

- Multi-stage Dockerfile.
- Pinned supported base.
- Non-root user.
- Read-only-compatible runtime.
- Health endpoint.
- Build and runtime tests.
- SBOM and vulnerability report.
- Version and source labels.
- Documented configuration, ports, volumes and shutdown behavior.

Hands-on lab

265. Select or build a small application.
266. Create the image.
267. Measure size and build time.
268. Run security and behavior tests.
269. Publish to a local or private registry.
270. Produce a release and rollback note.

Knowledge check

271. Can the image run without root?
272. Is every dependency justified?
273. Can another engineer reproduce the build?

Completion evidence: _____

Unit 46: Capstone 2: Multi-Service Compose Platform

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Design a multi-tier stack.
- Implement networking and persistence.
- Operate failures and recovery.

Why this matters

This capstone integrates services, networks, volumes, health, security and observability.

Scenario

Create a frontend, API, database and optional reverse proxy or monitoring service.

Mandatory deliverables

- Separate edge and backend networks.
- Persistent database volume.
- Health checks and retries.
- No unnecessary host exposure.
- Secrets outside images.
- Backup and restore procedure.
- Failure simulations.
- Operations runbook.

Hands-on lab

- 274. Build the Compose project.
- 275. Validate service-name DNS.
- 276. Replace every container without data loss.
- 277. Simulate DNS, health, permission and disk failures.
- 278. Back up and restore data.
- 279. Document evidence.

Knowledge check

- 280. Is the database publicly exposed?
- 281. Can the stack recover from restart?
- 282. Does the runbook identify each layer?

Completion evidence: _____

Unit 47: Docker Interview Mastery

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Answer beginner through senior questions.
- Explain tradeoffs.
- Solve production scenarios.

Why this matters

Interviews test mental models, safe operation and architecture - not only memorized commands.

Core questions

- Container versus VM.
- Image versus container.
- How layers and caching work.
- CMD versus ENTRYPOINT.
- Volume versus bind mount.
- EXPOSE versus publish.
- Bridge versus host networking.
- Why containers exit.
- How to reduce image size.
- How to run as non-root.
- How to handle secrets.
- How to debug an unreachable service.
- How to patch a vulnerable image.
- How to promote an image to production.
- How to recover persistent data.

Answer framework

Clarify goal and environment, explain underlying state, identify risks, propose commands or design, define validation and rollback.

Hands-on lab

- 283. Write answers to all core questions.
- 284. Draw image, runtime, network and storage diagrams.
- 285. Practice three incident scenarios aloud.

Knowledge check

- 286. Do your answers explain why?
- 287. Do they include security and rollback?
- 288. Can you distinguish Docker and orchestrator behavior?

Completion evidence: _____

Unit 48: Final Practical Examination and Completion Review

Completion: [] Not started [] In progress [] Completed Date: _____

Learning objectives

- Demonstrate integrated Docker competence.
- Produce a portfolio.
- Identify and close skill gaps.

Why this matters

Completion means you can build, secure, operate and troubleshoot containerized systems independently.

Practical examination

- Install and validate Docker.
- Build an optimized multi-stage image.
- Run it with non-root and restricted privileges.
- Persist and restore data.
- Create isolated networks.
- Deploy a Compose stack.
- Push and pull from a registry.
- Build a CI test plan.
- Diagnose a broken service.
- Write a production runbook and rollback procedure.

Passing standard

Complete mandatory tasks, score at least 80% on theory questions, and provide reproducible evidence.

Hands-on lab

289. Complete the examination in a disposable environment.
290. Create a portfolio folder.
291. Repeat any weak unit.
292. Mark the final declaration.

Knowledge check

293. Can you rebuild from source?
294. Can you replace containers without losing data?
295. Can another engineer follow your documentation?

Completion evidence: _____

Master Completion Tracker

Unit	Title	Status	Completion date
1	Why Containers Exist	Not started	
2	Linux Foundations Behind Containers	Not started	
3	Installing Docker and Validating the Engine	Not started	
4	Docker CLI Object Model	Not started	
5	Running and Inspecting Containers	Not started	
6	Container Lifecycle and Signals	Not started	
7	Executing Commands, Logs and Copying Files	Not started	
8	Resource Limits and Runtime Controls	Not started	
9	Images, Layers and Digests	Not started	
10	Your First Dockerfile	Not started	
11	Dockerfile Instructions in Depth	Not started	
12	CMD, ENTRYPOINT and Signal-Safe Execution	Not started	
13	Layer Caching and Build Performance	Not started	
14	Multi-Stage Builds	Not started	
15	Base Image Selection	Not started	
16	Package Installation and Image Hygiene	Not started	
17	.dockerignore and Build Context Security	Not started	
18	BuildKit Advanced Features	Not started	
19	Container Filesystems and Writable Layers	Not started	
20	Volumes and Persistent Data	Not started	
21	Bind Mounts and Host Integration	Not started	
22	tmpfs and In-Memory Data	Not started	
23	Docker Networking Fundamentals	Not started	
24	User-Defined Networks and DNS	Not started	
25	Port Publishing and Exposure	Not started	
26	Network Drivers: Bridge, Host, None and Overlay	Not started	
27	Container Networking Troubleshooting	Not started	
28	Storage Troubleshooting and Data Recovery	Not started	
29	Docker Compose Fundamentals	Not started	
30	Compose Services, Dependencies and Health Checks	Not started	
31	Compose Configuration, Environment and Profiles	Not started	
32	Compose Networking and Storage	Not started	
33	Registries and Image Distribution	Not started	
34	Image Tagging and Release Promotion	Not started	
35	Container Security Foundations	Not started	
36	Docker Daemon and Host Security	Not started	
37	Image Vulnerability Scanning and SBOMs	Not started	
38	Secrets Management	Not started	
39	Supply-Chain Security and Image Trust	Not started	
40	Docker in CI Pipelines	Not started	
41	Testing Container Images	Not started	
42	Production Deployment Patterns	Not started	
43	Observability for Containers	Not started	
44	Production Troubleshooting and Maintenance	Not started	
45	Capstone 1: Production-Grade Container Image	Not started	
46	Capstone 2: Multi-Service Compose Platform	Not started	

47	Docker Interview Mastery	Not started	
48	Final Practical Examination and Completion Review	Not started	

Final Certification Checklist

- ☐ All 48 units completed.
- ☐ Every mandatory command laboratory reproduced.
- ☐ Capstone 1 completed with image tests and release evidence.
- ☐ Capstone 2 completed with backup, restore and failure simulations.
- ☐ Interview questions practiced aloud.
- ☐ Final practical examination score is at least 80%.
- ☐ Weak units were repeated and revalidated.

Personal Completion Declaration

I confirm that I completed the practical work and understand that real production container changes require authorization, peer review, tested backups, security controls, observability and rollback planning.

Name: _____ Date: _____
Signature: _____